

**ĐẠI HỌC ĐÀ NẴNG**  
**TRƯỜNG ĐẠI HỌC BÁCH KHOA**  
**KHOA CÔNG NGHỆ THÔNG TIN**

Tel. (+84.0236) 3736949, Fax. (84-511) 3842771

Website: <http://dut.udn.vn/khoacntt>, E-mail: [cntt@dut.udn.vn](mailto:cntt@dut.udn.vn)



**BÁO CÁO TUẦN**  
**Thực tập tốt nghiệp - Tuần 3**

| Họ tên sinh viên    | Mã sinh viên |
|---------------------|--------------|
| Nguyễn Văn Hoài Nam | 102200274    |
| Nguyễn Hoàng Anh    | 102200009    |

*Đà Nẵng, 11/2024*

## 1. Yêu cầu trong tuần

- Xây dựng hệ thống gồm 3 thiết bị Raspberry đã cài đặt sẵn WebOS kết nối qua LAN
- Xây dựng các chương trình python để gửi/ nhận các thông điệp bằng IPv4 (Unicast, Broadcast và Multicast) //Multicast sẽ sử dụng giao thức IGMP
- Xây dựng các chương trình python để gửi/ nhận các thông điệp bằng IPv6 (Unicast, Multicast, Anycast)
- Đánh giá hiệu năng và an toàn để đưa ra sự lựa chọn khi gửi/nhận thông điệp.

## 2. Báo cáo công việc

### a. IPv4

[12/12]

Kết nối Unicast v4

```
from scapy.all import *

#IPv4
SRC_IPv4 = "20.20.20.20"
DST_IPv4 = "10.10.22.236"

# Ports
VALID_SPORT = 13400
VALID_DPORT = 13400
pro_type = TCP

payload = "echo 'Unicast to 10.10.22.236'"

sendp(Ether()/IP(src=SRC_IPv4,dst=DST_IPv4)/pro_type(sport=VALID_SPORT, dport=VALID_DPORT)/payload)
```

## Code nhận packet

```
from scapy.all import *
import subprocess

def sniff_fnc(packet):
    if packet.haslayer(Raw) and packet.haslayer(IP) and
packet[IP].src=="20.20.20.20":
# Decode payload
        decoded_payload = packet[Raw].load.decode("utf-
8")
# Exec payload
        result = subprocess.run(decoded_payload,
shell=True, capture_output=True, text=True)
        print(result.stdout)

print("Starting listen...")

sniff(prn=sniff_fnc, store=0)
```

## Kết quả

```
selunes@selunes:~$ ssh root@10.10.22.236
root@raspberrypi4-64:~# ls
MBC          get-pip.py      multi.py        sender_scapy.py
d3_receive.py leavemulti.py   receiver_scapy.py test.py
root@raspberrypi4-64:~# vi receiver_scapy.py
root@raspberrypi4-64:~# python3 receiver_scapy.py
Starting listen...
Unicast to 10.10.22.236!
```

## Kết nối sử dụng broadcast v4

```
from scapy.all import *

#IPv4
SRC_IPv4 = "20.20.20.20"
DST_IPv4 = "255.255.255.255"

# Ports
VALID_SPORT = 13400
VALID_DPORT = 50021
pro_type = UDP
```

```
payload = "echo 'Broadcast to everyone!'"

sendp(Ether()/IP(src=SRC_IPv4,dst=DST_IPv4)/pro_type(sport=VALID_SPORT, dport=VALID_DPORT)/payload)
```

Kết quả: không nhận được gói tin

```
root@raspberrypi4-64:~# python3 receiver_scapy.py
Starting listen...
```

Kiểm tra trên Wireshark thấy gói tin được đưa đến switch/router, tuy nhiên sau đó không phát hiện bất kì gói tin nào từ router tới các máy đích

```

▶ Frame 424: 71 bytes on wire (568 bits), 71 bytes captured (568 bits) on interface wlp2s0, id 0
▶ Ethernet II, Src: Intel_c6:be:3e (84:ef:18:c6:be:3e), Dst: Cisco_b7:11:3f (1c:df:0f:b7:11:3f)
▶ Internet Protocol Version 4, Src: 20.10.2.12, Dst: 255.255.255.255
▼ User Datagram Protocol, Src Port: 13400, Dst Port: 50021
    Source Port: 13400
    Destination Port: 50021
    Length: 37
    Checksum: 0x9ec8 [unverified]
    [Checksum Status: Unverified]
    [Stream index: 3]
    [Stream Packet Number: 2]
    [Timestamps]
    UDP payload (29 bytes)
▶ [Malformed Packet: DoIP]
```

Khi kiểm tra, phát hiện có một số gói tin được đánh dấu broadcast, kiểm tra thì thấy địa chỉ MAC đích được set là ff:ff:ff:ff:ff:ff

```

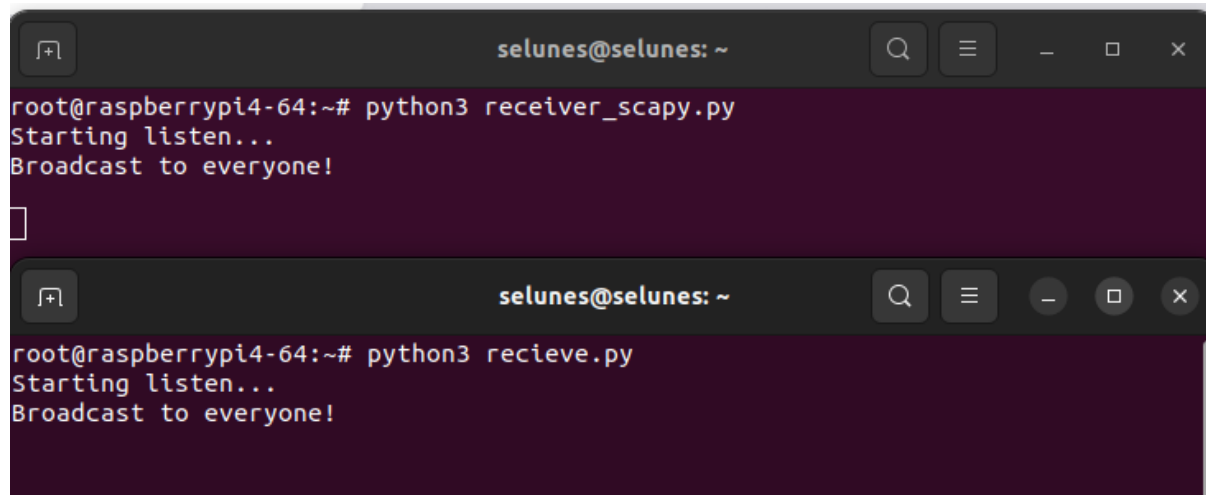
▶ Frame 3452: 243 bytes on wire (1944 bits), 243 bytes captured (1944 bits) on interface wlp2s0, id 0
▼ Ethernet II, Src: WebTVNETWORK_7e:64:56 (00:e0:1d:7e:64:56), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
    ▶ Destination: Broadcast (ff:ff:ff:ff:ff:ff)
    ▶ Source: WebTVNETWORK_7e:64:56 (00:e0:1d:7e:64:56)
    Type: IPv4 (0x0800)
    [Stream index: 37]
▶ Internet Protocol Version 4, Src: 10.10.22.51, Dst: 10.10.22.255
▼ User Datagram Protocol, Src Port: 138, Dst Port: 138
    Source Port: 138
    Destination Port: 138
    Length: 209
    Checksum: 0x876b [unverified]
    [Checksum Status: Unverified]
    [Stream index: 78]
    [Stream Packet Number: 1]
    [Timestamps]
    UDP payload (201 bytes)
▶ NetBIOS Datagram Service
▶ SMB (Server Message Block Protocol)
▶ SMB MailSlot Protocol
▶ Microsoft Windows Browser Protocol
```

Thực hiện set địa chỉ MAC thành ff:ff:ff:ff:ff:ff và gửi lại gói tin

```
DST_MAC = "FF:FF:FF:FF:FF:FF"
```

```
sendp(Ether(dst=DST_MAC)/IP(src=SRC_IPv4,dst=DST_IPv4)/pr  
o_type(sport=VALID_SPORT, dport=VALID_DPORT)/payload)
```

Kết quả: đã nhận được gói tin



The image shows two terminal windows from a Raspberry Pi 4. The top window shows the execution of `python3 receiver_scapy.py`, which outputs "Starting listen..." and "Broadcast to everyone!". The bottom window shows the execution of `python3 recieve.py` (note the typo), which also outputs "Starting listen..." and "Broadcast to everyone!". Both windows have a title bar indicating the user is `selunes` on a host named `selunes`.

Về multicast, các nhóm đã thử tìm cách setting các group multicast, sau đó đưa ra kết luận: không thể config trực tiếp các group thông qua terminal mà phải thông qua code, cụ thể với python là package socket.

[13/12]

Code tham gia multicast v4 group và nhận gói tin

```
import socket
import struct

MCAST_GRP = '224.12.1.1' # Insert group
MCAST_PORT = 50021

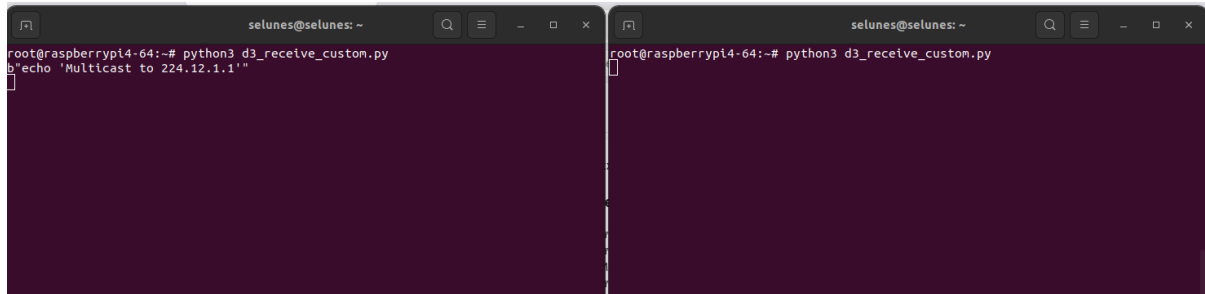
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM,
socket.IPPROTO_UDP)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR,
1)

sock.bind((MCAST_GRP, MCAST_PORT))

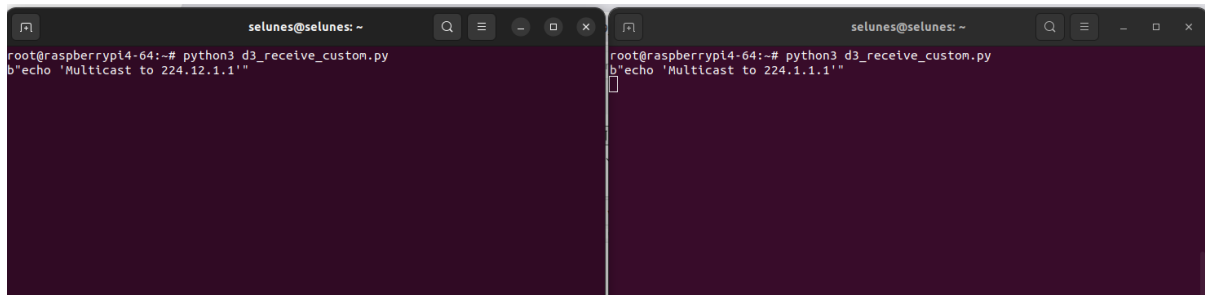
mreq = struct.pack("4sl", socket.inet_aton(MCAST_GRP),
socket.INADDR_ANY)
sock.setsockopt(socket.IPPROTO_IP,
socket.IP_ADD_MEMBERSHIP, mreq)
```

```
while True:
    print(sock.recv(10240))
```

Thực hiện gửi gói tin tới multicast IP 224.12.1.1, thu được kết quả



Thực hiện gửi gói tin tới multicast IP 224.1.1.1, thu được kết quả



Để xác nhận phương thức sử dụng là IGMP, thực hiện chuyển code tham gia multicast và nhận gói tin lên máy Linux có kết nối mạng. Kết quả

- Khi bắt đầu chạy code, có 2 packet IGMP với info là Membership Report group => đã tham gia vào group multicast

| No.  | Time         | Source       | Destination | Protocol | Length | Info                              |
|------|--------------|--------------|-------------|----------|--------|-----------------------------------|
| 2014 | 86.761400855 | 10.10.22.106 | 224.1.1.1   | IGMPv2   | 46     | Membership Report group 224.1.1.1 |
| 2061 | 91.384443231 | 10.10.22.106 | 224.1.1.1   | IGMPv2   | 46     | Membership Report group 224.1.1.1 |

- Khi terminate code, xuất hiện 2 packet, một packet thông báo tới địa chỉ 224.0.0.2 rằng máy đã rời group, packet còn lại là query lại thành viên trong group

|       |               |              |           |        |    |                                                |
|-------|---------------|--------------|-----------|--------|----|------------------------------------------------|
| 11647 | 254.956953134 | 10.10.22.106 | 224.0.0.2 | IGMPv2 | 46 | Leave Group 224.1.1.1                          |
| 11649 | 254.978349794 | 0.0.0.0      | 224.0.0.1 | IGMPv2 | 52 | Membership Query, specific for group 224.1.1.1 |

## b. IPv6

Đối với IPv6, nhóm sử dụng package socket thay vì scapy

[13/12]

Code gửi Unicast IPv6

```
import socket
```

```
with socket.socket(socket.AF_INET6, socket.SOCK_DGRAM) as
s:
s.sendto(b"Unicast to abcd:1234::5", ("abcd:1234::5",
23232))
```

Code nhận gói tin

```
import socket
print("Waiting for a message...")

while True:
with socket.socket(socket.AF_INET6, socket.SOCK_DGRAM) as
s:
s.bind( "::", 23232))

data, addr = s.recvfrom(1024)
print(f"Received message: {data.decode()} from {addr}")
```

Kết quả:

```
root@raspberrypi4-64:~# vi receiver_v6.py
root@raspberrypi4-64:~# python3 receiver_v6.py
Waiting for a message...
Received message: Unicast to abcd:1234::5 from ('abcd:1234::10', 41728, 0, 0)
□
```

Code gửi Multicast IPv6

```
import socket

with socket.socket(socket.AF_INET6, socket.SOCK_DGRAM) as
s:
s.sendto(b"Multicast to ff02::12:1:1", ("ff02::12:1:1",
23232))
```

Code tham gia Multicast và nhận gói tin IPv6

```
import socket
import struct
MCAST_GRP = 'ff02::12:1:1' # Insert multicast group here
MCAST_PORT = 23232
```

```

sock = socket.socket(socket.AF_INET6, socket.SOCK_DGRAM,
socket.IPPROTO_UDP)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR,
1)

sock.bind(('', MCAST_PORT))

mreq = struct.pack("16sI",
socket.inet_pton(socket.AF_INET6, MCAST_GRP), 0)
sock.setsockopt(socket.IPPROTO_IPV6,
socket.IPV6_JOIN_GROUP, mreq)

while True:
print(sock.recv(10240))

```

Thực hiện gửi gói tin lần lượt đến multicast IP ff02::12:1:1 và ff02::1:1:1, thu được kết quả

```

selunes@selunes: ~
root@raspberrypi4-64:~# python3 d3_receive_custom.py
b'Multicast to ff02::1:1:1'

selunes@selunes: ~
root@raspberrypi4-64:~# python3 d3_receive_custom.py
b'Multicast to ff02::12:1:1'

selunes@selunes: ~
root@raspberrypi4-64:~# python3 d3_receive_custom.py
b'Multicast to ff02::12:1:1'

selunes@selunes: ~
root@raspberrypi4-64:~# python3 d3_receive_custom.py
b'Multicast to ff02::1:1:1'

```

Code gửi Anycast IPv6

```

import socket

with socket.socket(socket.AF_INET6, socket.SOCK_DGRAM) as
s:
s.sendto(b"Anycast to abcd:1234::5", ("abcd:1234::5",
23232))

```

Kết quả



```
selunes@selunes: ~  
RX packets:1023 errors:0 dropped:0 overruns:0 frame:0  
TX packets:1023 errors:0 dropped:0 overruns:0 carrier:0  
collisions:0 txqueuelen:1000  
RX bytes:90645 (88.5 KiB) TX bytes:90645 (88.5 KiB)  
  
wlan0 Link encap:Ethernet HWaddr D8:3A:D0:A4:BF:C0  
inet addr:10.10.22.236 Bcast:10.10.22.255 Mask:255.255.255.0  
inet6 addr: fe80::da3a:ddff:fea4:bfc0/64 Scope:Link  
inet6 addr: abcd:1234::5/64 Scope:Global  
UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1  
RX packets:52346 errors:0 dropped:2081 overruns:0 frame:0  
TX packets:17291 errors:0 dropped:0 overruns:0 carrier:0  
collisions:0 txqueuelen:1000  
RX bytes:17462712 (16.6 MiB) TX bytes:1947891 (1.8 MiB)  
  
root@raspberrypi4-64:~# python3 receiver_v6.py  
Waiting for a message...  
Received message: Anycast to abcd:1234::5 from ('abcd:1234::10', 57292, 0, 0)  
Received message: Anycast to abcd:1234::5 from ('abcd:1234::10', 38495, 0, 0)  
Received message: Anycast to abcd:1234::5 from ('abcd:1234::10', 55352, 0, 0)  
Received message: Anycast to abcd:1234::5 from ('abcd:1234::10', 57283, 0, 0)  
Received message: Anycast to abcd:1234::5 from ('abcd:1234::10', 41607, 0, 0)  
Received message: Anycast to abcd:1234::5 from ('abcd:1234::10', 51850, 0, 0)  
  
selunes@selunes: ~  
RX bytes:2700875 (2.5 MiB) TX bytes:473827 (462.7 KiB)  
  
lo Link encap:Local Loopback  
inet addr:127.0.0.1 Mask:255.0.0.0  
inet6 addr: ::1/128 Scope:Host  
UP LOOPBACK RUNNING MTU:65536 Metric:1  
RX packets:594 errors:0 dropped:0 overruns:0 frame:0  
TX packets:594 errors:0 dropped:0 overruns:0 carrier:0  
collisions:0 txqueuelen:1000  
RX bytes:49508 (48.3 KiB) TX bytes:49508 (48.3 KiB)  
  
wlan0 Link encap:Ethernet HWaddr D8:3A:D0:A4:BE:B9  
inet addr:10.10.22.237 Bcast:10.10.22.255 Mask:255.255.255.0  
inet6 addr: fe80::da3a:ddff:fea4:beb9/64 Scope:Link  
inet6 addr: abcd:1234::5/64 Scope:Global  
UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1  
RX packets:43065 errors:0 dropped:2052 overruns:0 frame:0  
TX packets:11162 errors:0 dropped:0 overruns:0 carrier:0  
collisions:0 txqueuelen:1000  
RX bytes:16702007 (15.9 MiB) TX bytes:1176952 (1.1 MiB)  
  
root@raspberrypi4-64:~# python3 receiver_v6.py  
Waiting for a message...
```

### c. Đánh giá hiệu năng và an toàn

- Unicast: Quá trình truyền tải nhanh gọn do gửi trực tiếp đến một đích cụ thể, tuy nhiên nếu muốn gửi gói tin tới nhiều đích khác nhau thì không hiệu quả
- Broadcast: Cho phép đơn giản hóa việc gửi gói tin đến tất cả thiết bị trong một mạng, tuy nhiên sẽ gây ra lượng băng thông lớn, trong một số trường hợp sẽ gây nghẽn mạng. Đồng thời, broadcast sẽ công khai thông tin nên dễ bị rủi ro bảo mật hơn
- Multicast: Cho phép đơn giản hóa việc gửi gói tin đến tất cả thiết bị trong một nhóm thiết bị. Giúp tiết kiệm băng thông hơn broadcast, tuy nhiên cần phải có cấu hình riêng trong mạng thông qua code hoặc router.
- Anycast: Giúp phân tán lưu lượng và cải thiện hiệu suất hệ thống do gói tin được chuyển tới đích gần nhất. Nếu đích gần nhất bị tấn công thì gói tin sẽ được chuyển hướng sang đích khác để đảm bảo không bị gián đoạn.